Zenith: Dated Evaluation and Controlled Updates of a Reasoning Graph

Implementation and retrospective evaluation

Will Cook*

Draft, 11 October 2026

Abstract

Reasoning graphs can combine data acquisition, language-model calls and output checks, yet a poor final prediction rarely identifies which component should change. Zenith implements a dated evaluation loop for such graphs. A native executor binds each run to its graph, node contracts, context policy, input snapshot and model receipts. A separate evaluator resolves price forecasts against exact later observations and retains incomplete support. Mechanical diagnostics describe recorded failures and context delivery; an agent proposes one bounded policy intervention from development evidence. The same executor and evaluator compare the candidate with a frozen baseline. We describe this implementation, its conditional formal contracts and a chronological assessment protocol. A compact historical study completed 100 reasoning calls. A fixed persistence/scale rule lowered model-baseline loss in both windows; on five later dates its mean relative price error was 0.8875%, versus 0.9966% for the model baseline and 1.0453% for no change. These exposed historical windows establish native pipeline feasibility and a descriptive comparison. Forecasting advantage, generalized graph learning and novelty remain unestablished.

1 Problem and scope

A financial forecast is issued with a particular information set and judged after its horizon matures. If its target price is inaccurate, possible explanations include unavailable source evidence, evidence excluded from a node's context, an unsupported inference, model-response variation and an unpredictable outcome. The final error does not distinguish these explanations. The relevant experiment changes a declared part of the process and generates new predictions on matched evidence.

Zenith extracts an existing Lab–Oracle–Refinement system into a standalone repository. The authored inventory contains 71 nodes; the default Lab plan contains 28 nodes in 11 execution waves. These are structural counts, not completed forecasts. Lab consumes forward evidence and emits structured thesis and forecast checkpoints, called CP1 and CP2. Oracle inspects later observations and prepares hindsight feedback. The retained refinement runner and the newer reflection/cohort route are separate implementations.

*AI agents researched and drafted this manuscript under Will Cook's direction. The source and experimental receipts define the evidence reported here.

Graph of Thoughts already represents reasoning operations, aggregation and scoring as graphs [3]. Trace, TextGrad, GEPA, AFlow and ADAS already study optimization of compound systems or generated workflows [1, 5, 9, 21, 22]. The implementation described here combines dated evidence admission, native contracts, complete forecast accounting and reviewed interventions. We make no claim that this combination is new or that it outperforms those methods.

2 Method

2.1 One executor and a frozen run identity

A node declares dependencies, instructions and an output schema. Frozen reasoning policies can add a short skill rule, set nonnegative incoming-edge weights and impose a context-character budget. The base instruction and output contract remain binding. The executor preserves canonical upstream artifacts; weighted context views retain either the full serialized artifact, a declared excerpt or an omission marker. A receipt records source hashes, original and retained character counts, and missing dependencies.

The dependency graph describes scheduled node inputs. Effective requests also contain runtime-injected contracts, temporal substitutions and other configured context. Prompt hashes and request identities are recorded separately. Trace availability depends on the provider: agent-provider request packets retain complete prompts, while the budgeted paid adapter records prompt hashes and the engine's attempt sidecars retain a prompt snippet of at most 4,000 characters alongside response text. Declared ancestry therefore identifies where to inspect; it does not establish actual model consumption or causal responsibility.

The native engine can pause for an agent response. A request packet binds the run, node and prompt hash. Imported responses pass through the original output validation and resume path. Explicit provider adapters record requested and actual model identity, attempts and reported usage. Candidate evaluation uses these same execution and validation paths. Paid cohorts require an identical requested provider endpoint in both arms; receipts check the returned provider against the endpoint's catalog binding.

Topology variants have their own admission rules. Adding a check or changing a connection must preserve the relevant output contracts and temporal boundary. An explicitly admitted compact graph is a separate experimental design; results from it do not establish execution or usefulness of the full default Lab.

2.2 Dated evidence and forecast measurement

Observation time, first availability time and acquisition time are distinct. For issue time t, forward input must belong to the subject-side evidence class and have recorded availability no later than t. Unknown availability remains an uncertainty. A declared conservative release rule can support historical replay, but it cannot prove the original publication or acquisition sequence. Future truth is kept in a separate evaluator.

Every forecast specifies a target, horizon, issue price, target price, price basis and currency. Before the horizon matures, its evaluation is pending. Missing, duplicated or

incompatible target observations produce a data gap without a realized error. A missing evaluation target is distinct from missing evidence in the forward snapshot.

For a complete issue i with fixed target set S_i, target prices q_{is} and positive realized prices y_{is}, the native cohort loss is

$$L_i = \frac{1}{|S_i|} \sum_{s \in S_i} \frac{|q_{is} - y_{is}|}{y_{is}}, \qquad L = \frac{1}{n} \sum_{i=1}^{n} L_i.$$

This gives each issue equal weight even when target counts differ. The evaluator also reports direction errors and a no-change forecast using the issue price. Price error is not a trading return, probability-calibration score or causal diagnosis. Different horizon objectives require separately declared comparisons or weights fixed before selection.

The cohort importer checks frozen inputs, graph and policy identities, implementation hashes, declared horizon and model receipts. A comparison requires every candidate to have the complete preregistered issue/target support. If any run or truth observation remains incomplete, aggregate and paired scores remain unavailable. For complete support it records $L_i^{\text{candidate}} - L_i^{\text{baseline}}$; negative values favor the candidate. Coverage and failed attempts remain visible.

2.3 Financial task and loss

This experiment forecasts a next-session price, rather than selecting trades or estimating portfolio returns. A low price-level error can coexist with no advantage over the current-price forecast. No-change and past-one-session momentum references therefore use the same issue/target support and loss as the model arms. Testing market efficiency would additionally require a specified information set and risk-adjusted returns after trading costs; the present harness does not perform that test [14].

The implemented loss is absolute percentage error without the factor of 100. For positive outcomes with finite inverse moment, its optimal point is an inverse-price-weighted median, generally distinct from the conditional mean [8]. The current instructions request a target price without specifying that functional. A subsequent protocol should disclose its loss or request a statistical functional with a compatible score before execution. Percentage losses are unsuitable for zero or signed return targets; a training-scaled absolute-error objective is a possible separately preregistered alternative, provided its training denominator is nonzero [10]. The present receipts retain their original price loss.

2.4 Diagnostics and reviewed proposals

The diagnostic observer reads run manifests, artifacts, handoff packets, validation records and context-policy receipts. It distinguishes absent records from unreadable or mismatched records. Its verification labels identify checks of recorded bytes, hashes and declared bindings. It does not independently certify execution, source timestamps, model behavior or consumption of context. Observer failures cannot change cohort acceptance.

Feedback aggregation retains successes, failures and unresolved denominators. Its default proposal gate requires at least 20 distinct issue times and recurring failures

Table 1: Diagnostic evidence and its reach. These observations may select a trace for investigation; they do not assign causal blame.

Recorded condition	Interpretation
Artifact absent	No artifact exists at the declared node's expected path within the run.
Artifact or receipt unbound	The purported evidence remains unknown.
Native validation rejection	A bound record reports rejection by the native validator.
Context missing, omitted or excerpted	A bound policy receipt records a delivery condition.
Forecast price/direction error	Exact later truth supports the measured discrepancy.

across three time buckets. These counts route repeated-error investigations; they are not a significance test.

A development reflection packet binds the feedback proposal, membership, sources and a bounded selection of verified node artifacts and available agent-provider handoff exchanges. It does not consume the paid engine's debug sidecars; without a handoff packet, its selected-node input prompt remains unavailable. An agent supplies a hypothesis and one proposed change. The compiler checks the measured graph and baseline policy, then changes one short skill rule, one context budget or one incoming-edge weight through the existing policy validator. Its output is an ordinary native policy with an exact before/after record and evidence hashes. Compilation neither executes nor accepts the candidate.

A useful first intervention restores a clipped context view while preserving canonical evidence and the skill. A skill intervention instead holds the input view fixed. New data acquisition needs a separately declared input-ablation protocol; silently substituting source snapshots would confound the comparison. When two changes appear coupled, a small preregistered baseline/A/B/A+B study can test their interaction. Single-component search has no general optimality guarantee.

3 Contract checking

The Lean model treats a skill as a parameterized input/output relation. Its composition result is conditional: if the first skill establishes an intermediate condition and the second preserves the desired postcondition, their composition satisfies that postcondition. Other results express subject-side availability constraints and rank-based acyclicity. An executable finite rank certificate can instantiate the graph theorem.

The Python checker verifies declared port types, evidence classes, source hashes and finite ranks. It explicitly leaves behavioral semantics unverified. These checks do not prove that a natural-language instruction implements its declared relation. They also do not prove timestamp authenticity, successful reasoning or forecast accuracy. Source

and checking entrypoints are listed in Appendix A; completed checking receipts must identify their exact inputs.

4 Evaluation protocol and evidence

4.1 Chronological assessment

Candidate generation and selection are part of fitting [4]. An inner development window supplies failures and proposals; later inner validation compares a fixed candidate bank. Before an outer assessment, freeze the selected design, selection procedure, evaluator and resource budget. Development labels must mature before outer issue times, with a declared gap and exclusion of overlapping label intervals. Group assets by issue time.

The implemented cohort freezes candidates and shared inputs, separates development and holdout membership by calendar week, checks development target times plus an embargo against the first heldout issue, and seals final assessment before opening it. Its boundary permits equality after the declared embargo; a study must specify whether an observation finalized at the issue time was already available. A conservative protocol excludes that ambiguous boundary.

Nested rolling coordination and exposure accounting across studies remain planned. Renaming a study cannot restore an unseen assessment. Adaptive holdout research motivates this distinction, but its assumptions and protected query mechanisms are not implemented here [6]. Once inspected, an outer period becomes development history. Fresh prospective predictions are needed to assess a procedure that has repeatedly searched the same history.

Historical LLM evaluation also needs a separate check on pretrained knowledge. Glasserman and Lin compare original and anonymized news headlines, treating company knowledge as both potential look-ahead information and distraction [7]. KTD-Fin masks tickers and dates consistently through prompts and tools, probes re-identification, and separates portfolio returns into common, style and residual components [23]. These are relevant controls, not mechanisms currently implemented in Zenith. Masking can also change useful context, and limited re-identification tests cannot prove absence of memorization. The historical StockNet snapshot is not an uncontaminated model test merely because its explicit inputs precede its target.

Repeat an unchanged baseline when estimating model-response variation. Compare equal declared call, token and cost budgets, retain invalid and failed attempts, and preregister any simplicity tie-breaker. Neither a lower development mean nor 20 issues establishes statistical confidence. Statistical uncertainty must account for serial dependence and overlapping horizons. White's Reality Check evaluates a searched family against a benchmark using dependence-aware resampling [16]. It motivates preserving all paired losses and tried alternatives, but its assumptions and inferential procedure are not implemented here. The incomplete pilots supply no test statistic.

4.2 Current empirical evidence

This draft describes implemented execution, diagnostics and comparison interfaces. Synthetic regression fixtures establish software behavior under constructed inputs; they cannot establish forecasting improvement. Historical StockNet preparations and an incomplete input-comparison attempt likewise remain distinct from a complete native graph experiment [18].

Four compatibility pilots used the 1 July 2015 snapshot, excluded from both development and later assessment. The first two ran predictor profile v1; all 105 recorded implementation hashes match commit `c4c051c`. They used the unchanged native validator and no automatic retry. Requested and returned model identifiers agreed: `amazon/nova-micro-v1` and `qwen/qwen3.5-flash-02-23`, respectively. The third requested `deepseek/deepseek-v4.1-flash` under prompt-only profile v2, introduced at `7582bf1`, which requests word-count margins while retaining the validator. Its 105 implementation hashes match `aa2f8ec`; the introduction commit matches only 104, differing in the paid adapter. An HTTP 429 occurred at the first CP1 request; no model response, usage or actual model identity was recorded. The fourth pinned that same model to `deepinfra/fp8`; the returned provider was DeepInfra. Both CP1 and CP2 were accepted under the unchanged validator, with all 105 implementation hashes matching `01522cc`. These matches reconstruct implementation content: the manifests contain no observed Git HEAD, and a match need not identify a unique revision. These calibrations changed model, prompt or provider settings; they are not matched model-performance comparisons.

Table 2: Compatibility calibration, not forecast evaluation. Rejections report native output-contract checks. Costs are provider-reported USD.

Model	Attempts	Native outcome	Reported cost
Nova Micro	1	CP1 rejected: thesis 36 words (minimum 150); unbracketed reference IDs.	0.000307895
Qwen Flash	2	CP1 accepted; CP2 rejected: rationale 145 words (minimum 165).	0.001489930
DeepSeek Flash	1	HTTP 429 at CP1; no response; CP2 not started.	Unknown
DeepSeek/DeepInfra	2	CP1 and CP2 accepted; three-node run completed.	0.002375240

The first three pilots produced three responses from four attempts, with USD 0.001797825 in known reported charges and one unknown-cost attempt. Following the HTTP 429, an explicitly authorized, hash-bound ledger recovery retained that failed attempt, its full reservation and the original budget limits. The fourth pilot used a fresh run rather than resending the failed request. Its two responses cost USD 0.002375240; CP1 recorded 5,018 input/652 output tokens and CP2 recorded 6,272 input/1,240 output tokens. At compatibility completion the six attempts had five responses, USD 0.004173065 in known charges, one unknown charge and USD 0.038015120 in reservations. The ledger was no longer halted; reservations remain distinct from reported

charges. All failures remain recorded. Native acceptance demonstrates compatibility and pipeline execution, without measuring forecasting accuracy. Length/format rejection and transport failure likewise do not identify a reasoning failure.

The first native study was prepared at source commit `01522cc`, with frozen cohort hash `bd5224387591` (full identity in the evidence export). It uses a compact two-reasoning-node CP1–CP2 design and the same engine/evaluator. Development completed 40 native episodes across 20 issue dates and four calendar weeks, with complete support in both arms. A frozen baseline is compared with one short CP2 skill intervention on dated StockNet price evidence. The rule uses current close as a persistence prior, separates direction from move size and shrinks projected movement toward zero when past-return scale evidence is weak. It preserves targets, quotes, horizons and grounding requirements. The rule is fixed before development scoring; it is not a learned or reflection-generated update. The protocol generator selects the first 20 common mature August 2015 issue dates for development and the first five October dates for later assessment, with AAPL, MSFT and GOOG as fixed targets. It supplies 20 sessions of past OHLCV observations and forecasts the next exchange-session close. The two arms share graph, data, model and 24,000-character context caps; their sole difference is the CP2 skill rule. The complete two-arm study used 100 reasoning calls. The graph is admitted under the explicit predictor-replacement protocol, separately from ordinary variants that retain base ancestry. The retained default Lab graph is not the evaluated architecture of this study.

The development dates were previously exposed in a flat input study; the later assessment dates were examined in deterministic benchmark work. This study freezes membership and candidate identity, but the later period is not a pristine test set. The experiment can establish native pipeline feasibility and a controlled historical comparison under its declared identities. It cannot establish an unseen-history or prospective forecasting advantage.

Table 3: Native cohort study. Values are percentages, $100L$, with equal issue/target weighting and three fixed targets per date. Deterministic references share the support but are not model arms. Later assessment is sealed; exact scores remain in the evidence export.

Phase	Dates	Baseline	Candidate	No change	Momentum
Development	20	2.1000	1.9142	1.8254	2.3756
Later assessment	5	0.9966	0.8875	1.0453	0.9038

The development candidate-minus-baseline mean issue delta was -0.00185743, with lower candidate loss on 14 dates and higher loss on six. The frozen rule classified it as a candidate proposal for later assessment. This is a descriptive selection outcome, not statistical confidence or a learned update. The candidate's 1.9142% loss remained above the no-change reference's 1.8254%; it establishes no advantage over persistence. Development had 80 attempts and 80 responses, all 40 episodes accepted, USD 0.0992369112 in reported charges and no unknown costs or failed attempts. All calls returned the requested `deepseek/deepseek-v4.1-flash` through reported DeepInfra, with `deepin-`

`fra/fp8` requested in both arms. The development report and exact per-issue/target accounting are bound in `paper/study-evidence.json`.

Later assessment completed all 10 episodes across five dates and two calendar weeks, with three targets per date and arm. It was sealed once under `native-deepinfra-final-v1`. Candidate-minus-baseline mean issue loss was −0.00109107, lower on three dates and higher on two. The candidate's 0.8875% mean loss was below the model baseline, no-change and momentum references on this support. The native decision remained `insufficient_issues`: five dates are below its 20-issue proposal gate, and the receipt grants no live mutation permission. Twenty attempts returned 20 responses at USD 0.0244918688, with no failed or unknown-cost study calls. Both arms retained the same requested and actual model/provider identities.

Across calibration and production, the settled ledger records 106 attempts, 105 responses, USD 0.1279018450 in known charges and the earlier unknown-cost HTTP 429. It retains USD 1.137895220 in reservations and is not halted; reservations are not actual charges. The production study itself had 100 responses and USD 0.1237287800 in reported charges. Its final report matches the sealed state receipt, and both phase exports preserve full common support. These are descriptive observations from a modern model on previously examined 2015 windows, with one manually frozen intervention and no unchanged-baseline replicates. They establish neither significance nor a learned improvement, prospective forecasting advantage or profitability. Historical evaluation remains vulnerable to model-training contamination even when forward input timestamps are valid.

5 Related work and adopted mechanisms

5.1 Financial agents

TradingAgents organizes specialist analysis, bullish/bearish research, trading and risk review [17]. QuantAgent's 2024 writer/judge loop retrieves prior experience while an outer market-data evaluation feeds signal code, scores and feedback into its knowledge base [15]. This is close conceptual prior art for the whole feedback loop; its alpha-factor task and information-coefficient/Sharpe evaluation differ from dated price forecasting.

FinCon compares consecutive training trajectories and propagates revised beliefs to selected analysts and a manager [20]. This is close prior art for using measured outcomes to revise particular agent instructions. Its evaluation concerns single-stock trading and portfolio tasks measured by returns, Sharpe and drawdown; those results are not comparable to Zenith's point-price loss. Zenith's present artifact binds proposals to recorded node evidence and admits a bounded intervention for native replay. An independent causal diagnosis, successful update procedure and forecasting advantage remain unestablished.

The reviewed FinRobot source implements typed strategy-parameter proposals, up to three in-sample backtests, selection and a later tail evaluation [2]. This is relevant executable prior art for a bounded propose/evaluate/select loop. It is later code, not an experimental finding of the 2024 platform paper [19]. Its return objective, shared split-boundary date and fallback behavior are not Zenith's comparison contract. Paper ver-

sions, code pins and this distinction are retained in `docs/finance-related-work.json`. The compact Zenith study needs no specialist ensemble to test its single declared rule; additional nodes need separately measured ablations.

5.2 Compound-system optimization

Trace and TextGrad expose execution/context-bearing criticism [5, 21]; GEPA and MIPRO propose and search prompt candidates [1, 12]. Zenith uses development feedback to compile one reviewed policy change for native comparison. Reverse textual feedback, population/Pareto search, categorical TPE, demonstrations and minibatching remain deferred. Criticism is not a numerical derivative, and successful final outputs do not certify every intermediate trace.

AFlow searches code-represented workflows, while ADAS uses a meta agent and archive to generate and evaluate agent programs [9, 22]. Zenith retains parent-relative proposals and lineage through admitted policies and graph variants. Unrestricted generated-program execution is outside the current method.

Evolution strategies and Agent Lightning use numerical optimization or reinforcement learning interfaces [11, 13] unavailable through this hosted inference adapter. Their update signals do not identify a unique cause of a failed forecast. Context parameters can still be compared through controlled native reruns.

Exact implementation pins, inspected paths and adoption decisions reside in `docs/architecture-sources.json`. The paper versions and later code versions are recorded separately. No cited upstream optimizer or training stack is a silently installed runtime dependency.

6 Limitations

The principal empirical boundary is incomplete evidence for forecasting advantage and transfer of the update procedure. A historical prompt can encode outcomes learned during model training. Conservative availability rules cannot recover unknown publication clocks, revisions or price vintages. Missing input, restricted context and reasoning may interact; a successful intervention establishes conditional sensitivity on measured support, without uniquely identifying the original cause.

Current comparisons are descriptive. Generalization requires independent assessment of the entire proposal/selection procedure, sufficient dependent-data uncertainty analysis and prospective evidence. Cross-study exposure accounting, nested rolling selection and autonomous optimization are unfinished. A compact pilot cannot establish the usefulness of every branch of the default graph. Candidate code is trusted local code, not a security sandbox. Client-side provider budgets and reported costs do not guarantee provider billing.

These limits leave a concrete research question: at a fixed resource budget, does a verified diagnostic packet produce interventions that improve later forecast error and coverage over unchanged, compact and simple-search baselines? The implementation supplies the experiment boundary; the answer requires controlled measurements.

A Artifacts and reproduction

The source repository contains the executor and schemas, not redistributed market text or private provider packets. The following owners permit inspection of the claims in this paper.

Operation	Source owner
Native execution and planning	`zenith/runtime.py`; `system/core/engine.py`
Skill/context policy	`system/lib/reasoning_policy.py`
Forecast admission and truth	`feedback/engine.py`
Matched comparison and seals	`research/cohort.py`
Recorded diagnostics	`research/diagnosis.py`
Proposal compilation	`research/reflection.py`; `research/reflection_cohort.py`
Conditional formal model	`formal/ZenithSemantics.lean`; `scripts/check_semantics.py`
Source/data decisions	`docs/architecture-sources.json`; `docs/data-sources.md`

From an editable source checkout, `python -m zenith validate` and `python -m zenith plan lab` inspect the authored graph without inference. The repository's reproduction guide owns environment and offline checking commands. Provider execution is explicit. Experiment reports must bind the frozen specification, implementation hashes, source inputs and model receipts; a code citation alone is insufficient to reproduce a measured comparison.

References

[1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. arXiv:2507.19457v2, 2025. URL https://arxiv.org/abs/2507.19457v2. Version read: 2507.19457v2.

[2] AI4Finance Foundation. FinRobot: strategy selection source snapshot, 2026. URL https://github.com/AI4Finance-Foundation/FinRobot/tree/2717499b8e30f242640af08c4ad9afd1113c2d45. Read 11 October 2026; pinned source, strategy_agent.py. This citation identifies reviewed code, not a paper-era reproduction.

[3] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of Thoughts: Solving Elaborate Problems with Large Language Models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690, 2024. doi: 10.1609/aaai.v38i16.29720. URL https://ojs.aaai.org/index.php/AAAI/article/view/29720.

[4] Gavin C. Cawley and Nicola L. C. Talbot. On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010. URL https://www.jmlr.org/papers/v11/cawley10a.html.

[5] Ching-An Cheng, Allen Nie, and Adith Swaminathan. Trace is the Next AutoDiff: Generative Optimization with Rich Feedback, Execution Traces, and LLMs. arXiv:2406.16218v2, 2024. URL https://arxiv.org/abs/2406.16218v2. Version read: 2406.16218v2.

[6] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. Generalization in Adaptive Data Analysis and Holdout Reuse. arXiv:1506.02629v2, 2015. URL https://arxiv.org/abs/1506.02629v2. Version read: 1506.02629v2.

[7] Paul Glasserman and Caden Lin. Assessing Look-Ahead Bias in Stock Return Predictions Generated By GPT Sentiment Analysis. arXiv:2309.17322v1, 2023. URL https://arxiv.org/abs/2309.17322v1. Version read: 2309.17322v1.

[8] Tilmann Gneiting. Making and Evaluating Point Forecasts. *Journal of the American Statistical Association*, 106(494):746–762, 2011. doi: 10.1198/jasa.2011.r10138. URL https://arxiv.org/abs/0912.0902v2. Methods read in author preprint arXiv:0912.0902v2.

[9] Shengran Hu, Cong Lu, and Jeff Clune. Automated Design of Agentic Systems. arXiv:2408.08435v2, 2024. URL https://arxiv.org/abs/2408.08435v2. Version read: 2408.08435v2.

[10] Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006. doi: 10.1016/j.ijforecast.2006.03.001. URL https://robjhyndman.com/papers/mase.pdf. Methods read in author preprint dated 2 November 2005.

[11] Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K. Qiu, and Yuqing Yang. Agent Lightning: Train ANY AI Agents with Reinforcement Learning. arXiv:2508.03680v1, 2025. URL https://arxiv.org/abs/2508.03680v1. Version read: 2508.03680v1.

[12] Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. arXiv:2406.11695v2, 2024. URL https://arxiv.org/abs/2406.11695v2. Version read: 2406.11695v2.

[13] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution Strategies as a Scalable Alternative to Reinforcement Learning. arXiv:1703.03864v2, 2017. URL https://arxiv.org/abs/1703.03864v2. Version read: 1703.03864v2.

[14] Allan Timmermann and Clive W. J. Granger. Efficient market hypothesis and forecasting. *International Journal of Forecasting*, 20(1):15–27, 2004. doi: 10.1016/S0169-2070(03)00012-8. URL https://econweb.ucsd.edu/~atimmerm/emh.pdf. Information-set and economic-value definitions read in author preprint.

[15] Saizhuo Wang, Hang Yuan, Lionel M. Ni, and Jian Guo. QuantAgent: Seeking Holy Grail in Trading by Self-Improving Large Language Model. arXiv:2402.03755v1, 2024. URL https://arxiv.org/abs/2402.03755v1. Version read: 2402.03755v1.

[16] Halbert White. A Reality Check for Data Snooping. *Econometrica*, 68(5):1097–1126, 2000. doi: 10.1111/1468-0262.00152. URL https://bashtage.github.io/kevinsheppard.com/files/teaching/mfe/advanced-econometrics/White.pdf.

[17] Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. TradingAgents: Multi-Agents LLM Financial Trading Framework. arXiv:2412.20138v7, 2024. URL https://arxiv.org/abs/2412.20138v7. Version read: 2412.20138v7, revised 3 June 2025.

[18] Yumo Xu and Shay B. Cohen. Stock Movement Prediction from Tweets and Historical Prices. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics* (*Volume 1: Long Papers*), pages 1970–1979. Association for Computational Linguistics, 2018. doi: 10.18653/v1/P18-1183. URL https://aclanthology.org/P18-1183/.

[19] Hongyang Yang, Boyu Zhang, Neng Wang, Cheng Guo, Xiaoli Zhang, Likun Lin, Junlin Wang, Tianyu Zhou, Mao Guan, Runjia Zhang, and Christina Dan Wang. FinRobot: An Open-Source AI Agent Platform for Financial Applications using Large Language Models. arXiv:2405.14767v2, 2024. URL https://arxiv.org/abs/2405.14767v2. Version read: 2405.14767v2.

[20] Yangyang Yu, Zhiyuan Yao, Haohang Li, Zhiyang Deng, Yupeng Cao, Zhi Chen, Jordan W. Suchow, Rong Liu, Zhenyu Cui, Zhaozhuo Xu, Denghui Zhang, Koduvayur Subbalakshmi, Guojun Xiong, Yueru He, Jimin Huang, Dong Li, and Qianqian Xie. FinCon: A Synthesized LLM Multi-Agent System with Conceptual Verbal Reinforcement for Enhanced Financial Decision Making. arXiv:2407.06567v3, 2024. URL https://arxiv.org/abs/2407.06567v3. Version read: 2407.06567v3.

[21] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic "Differentiation" via Text. arXiv:2406.07496v1, 2024. URL https://arxiv.org/abs/2406.07496v1. Version read: 2406.07496v1.

[22] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating Agentic Workflow Generation. arXiv:2410.10762v4, 2024. URL https://arxiv.org/abs/2410.10762v4. Version read: 2410.10762v4.

[23] Taojie Zhu, Wentao Zhao, Rui Sun, Beidi Luan, Jiacheng Lu, Sinuo Wang, Jing Li, Daxin Jiang, Yonghong He, and Zuo Bai. From Knowing to Doing: A Memory-Controlled Benchmark for LLM Trading Agents on Stock Markets. arXiv:2605.28359v1, 2026. URL https://arxiv.org/abs/2605.28359v1. Version read: 2605.28359v1.